МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
КАФЕДРА ИИТ

Лабораторная работа №6

По дисциплине «Современный платформы программирования»

Выполнил:
Студент 3 курса
группы ПО-12:
Ефимович С.Л.
Проверил:
Кулик А.Д.

Брест 2026

Цель: освоить приемы тестирования кода на примере использования пакета pytest

Ход работы

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)

1. Создайте файл test_cart.py. Реализуйте следующие тесты:
• Проверка добавления товара: после add_item("Apple", 10.0) в корзине должен быть один элемент.
• Проверка выброса ошибки при отрицательной цене.
• Проверка вычисления общей стоимости (total()).
2. Протестируйте метод apply_discount с разными значениями скидки:
• 0% - цена остаётся прежней
• 50% - цена уменьшается вдвое
• 100% - цена становится ноль
• < 0% и > 100% - должно выбрасываться исключение
Используйте @pytest.mark.parametrize
3. Создайте фикстуру empty_cart, которая возвращает пустой экземпляр Cart

```
@pytest.fixture
def empty_cart():
return Cart()
```

Используйте эту фикстуру в тестах, где нужно создать новую корзину.
4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:

```
import requests
def log_purchase(item):
requests.post("https://example.com/log", json=item)
```

• Замокайте requests.post, чтобы не было реального HTTP-запроса
• Убедитесь, что он вызывается с корректными данными
5. Добавьте поддержку купонов:

```
def apply_coupon(cart, coupon_code):
coupons = {"SAVE10": 10, "HALF": 50}
if coupon_code in coupons:
cart.apply_discount(coupons[coupon_code])
else:
```

raise ValueError("Invalid coupon")

• Напишите тесты на apply_coupon

• Замокайте словарь coupons с помощью monkeypatch или patch.dict

Код программы:

shopping.py

```
class Cart:
   def __init__(self):
       self.items = []

   def add_item(self, name, price):
       if price < 0:
           raise ValueError("Price cannot be negative")
       self.items.append({"name": name, "price": price})

   def total(self):
       return sum(item["price"] for item in self.items)

   def apply_discount(self, percent):
       if percent < 0 or percent > 100:
           raise ValueError("Invalid discount")

       for item in self.items:
           item["price"] *= 1 - percent / 100


import requests


def log_purchase(item):
   requests.post("https://example.com/log", json=item)
```

test_cart.py

```
import pytest
from shopping import Cart, log_purchase


@pytest.fixture
def empty_cart():
   return Cart()


def test_add_item(empty_cart):
   empty_cart.add_item("Apple", 10.0)
   assert len(empty_cart.items) == 1


def test_add_item_negative_price(empty_cart):
   with pytest.raises(ValueError):
       empty_cart.add_item("Apple", -5.0)


def test_total(empty_cart):
```

```
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.0)
    assert empty_cart.total() == 15.0


@pytest.mark.parametrize(
    "discount, expected",
    [
        (0, 100.0),
        (50, 50.0),
        (100, 0.0),
    ],
)
def test_apply_discount_valid(empty_cart, discount, expected):
    empty_cart.add_item("Item", 100.0)
    empty_cart.apply_discount(discount)
    assert empty_cart.total() == expected


@pytest.mark.parametrize("discount", [-10, 150])
def test_apply_discount_invalid(empty_cart, discount):
    empty_cart.add_item("Item", 100.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(discount)


def test_log_purchase(monkeypatch):
    called = {}

    def mock_post(url, json):
        called["url"] = url
        called["json"] = json

    monkeypatch.setattr("shopping.requests.post", mock_post)

    item = {"name": "Apple", "price": 10}
    log_purchase(item)

    assert called["url"] == "https://example.com/log"
    assert called["json"] == item
```

Результат работы программы:

```
test_cart.py .........
test_string_utils.py ........

============================================================================ warnings summary =======
.venv/lib/python3.9/site-packages/urllib3/__init__.py:35
  /Users/svyat/PythonProject/PythonProject/Lab6/.venv/lib/python3.9/site-packages/urllib3/__init__.
urrently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib
    warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=================================================================== 17 passed, 1 warning in 0.36s

(.venv) svyat@MacBook-Pro-Svat Lab6 %
```

Рисунок 1. Результат выполнения программы

Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Код программы:

```
import pytest
from main import Razmah
from task2 import maxPrefix, sortBubble



def test_razmah_normal():
   assert Razmah([6, 4, 2, 6, -2, 5, 8]) == 10


def test_razmah_one_element():
   assert Razmah([5]) == 0


def test_razmah_negative_numbers():
   assert Razmah([-10, -5, -1]) == 9


def test_razmah_empty():
   with pytest.raises(ValueError):
       Razmah([])


def test_razmah_none():
   with pytest.raises(TypeError):
       Razmah(None)



def test_maxprefix_normal():
   assert maxPrefix(["flower", "flow", "flight"]) == "f.l"


def test_maxprefix_no_common():
   assert maxPrefix(["dog", "racecar", "car"]) == ""


def test_maxprefix_one_string():
   assert maxPrefix(["abc"]) == "a.b.c"


def test_maxprefix_empty_strings():
   assert maxPrefix(["", ""]) == ""
```

```
def test_maxprefix_none():
   with pytest.raises(TypeError):
       maxPrefix(None)


def test_sortbubble_normal():
   assert sortBubble([3, 1, 4, 2]) == [4, 3, 2, 1]


def test_sortbubble_already_sorted():
   assert sortBubble([5, 4, 3]) == [5, 4, 3]


def test_sortbubble_with_duplicates():
   assert sortBubble([2, 2, 1]) == [2, 2, 1]


def test_sortbubble_strings():
   assert sortBubble(["a", "c", "b"]) == ["c", "b", "a"]


def test_sortbubble_empty():
   assert sortBubble([]) == []


def test_sortbubble_none():
   with pytest.raises(TypeError):
       sortBubble(None)
```

Результат работы программы:

```
test_lab1.py ................

========================================================================== 16 passed in 0.04s =

(.venv) svyat@MacBook-Pro-Svat Lab1SPP %
```

Рисунок 2. Результат выполнения программы

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

Код программы:

string_utils.py

```
def index_of_difference(str1, str2):
   if str1 is None and str2 is None:
       raise TypeError

   if str1 is None or str2 is None:
       return 0
```

```
    if str1 == "" and str2 == "":
        return -1

    min_len = min(len(str1), len(str2))

    for i in range(min_len):
        if str1[i] != str2[i]:
            return i

    if len(str1) != len(str2):
        return min_len

    return -1
```

test_string_utils.py

```
import pytest
from string_utils import index_of_difference


def test_none_none():
    with pytest.raises(TypeError):
        index_of_difference(None, None)


def test_empty_strings():
    assert index_of_difference("", "") == -1


def test_empty_and_nonempty():
    assert index_of_difference("", "abc") == 0
    assert index_of_difference("abc", "") == 0


def test_equal_strings():
    assert index_of_difference("abc", "abc") == -1


def test_difference_middle():
    assert index_of_difference("i am a machine", "i am a robot") == 7


def test_prefix_case():
    assert index_of_difference("ab", "abxyz") == 2


def test_difference():
    assert index_of_difference("abcde", "abxyz") == 2


def test_full_difference():
    assert index_of_difference("abcde", "xyz") == 0
```

Результат выполнения программы:

```
test_cart.py .........
test_string_utils.py ........

============================================================================ warnings summary ========
.venv/lib/python3.9/site-packages/urllib3/__init__.py:35
  /Users/svyat/PythonProject/PythonProject/Lab6/.venv/lib/python3.9/site-packages/urllib3/__init__.p
urrently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3
    warnings.warn(

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
=================================================================== 17 passed, 1 warning in 0.36s =

(.venv) svyat@MacBook-Pro-Svat Lab6 %
```

Рисунок 3. Результат выполнения программы

Вывод: приобрел практические навыки разработки API и баз данных